MERN Stack Bootcamp Lecture
Instructor: Kashif Raza | LinkedIn
Audience: Beginner to Intermediate React Developers
Duration: ~120 minutes

🎯 Lecture Objectives

By the end of this session, you will be able to:

1. **Identify** which hook to use for specific problems (state, effects, performance, refs, context)
2. **Implement** all major React hooks with confidence: `useState`, `useEffect`, `useCallback`, `useMemo`, `useReducer`, `useRef`, `useContext`
3. **Optimize** components using memoization hooks correctly—without premature optimization
4. **Build** complex, maintainable features by combining multiple hooks
5. **Avoid** common anti-patterns and performance pitfalls

> 💡 **Instructor Note:** This is your hooks reference manual. Bookmark it. We'll go deep on *when* and *why*—not just *how*.

📚 Hooks Overview: The Big Picture

Category	Hooks	Primary Use Case
State	`useState`, `useReducer`	Manage component data that changes over time
Effects	`useEffect`, `useLayoutEffect`	Handle side effects: API calls, subscriptions, DOM mutations
Performance	`useCallback`, `useMemo`	Prevent unnecessary re-renders and expensive recalculations
References	`useRef`	Access DOM nodes or persist mutable values without re-renders
Context	`useContext`	Consume context values without prop drilling
Advanced	`useImperativeHandle`, `useDebugValue`, Custom Hooks	Specialized use cases, library development, debugging

> 🔄 **Key Mental Model:** Every hook solves a specific problem. Don't reach for `useMemo` because it sounds advanced—use it only when you've measured a performance issue.

---

1. State Management Hooks

`useState`: Simple State for Functional Components

```
const [state, setState] = useState(initialValue);
```

✅ When to Use

- Primitive values: strings, numbers, booleans
- Simple objects/arrays that don't have complex update logic

- Local component state not shared elsewhere

✅ Real-World Example: Multi-Step Form

```
// hooks/useFormStep.js
import { useState } from 'react';

export function useFormStep(totalSteps) {
  const [currentStep, setCurrentStep] = useState(0);
  const [formData, setFormData] = useState({});
  const [errors, setErrors] = useState({});

  const nextStep = () => {
    if (currentStep < totalSteps - 1) {
      setCurrentStep(prev => prev + 1);
    }
  };

  const prevStep = () => {
    if (currentStep > 0) {
      setCurrentStep(prev => prev - 1);
    }
  };

  const updateField = (field, value) => {
    setFormData(prev => ({ ...prev, [field]: value }));
    // Clear error when user types
    if (errors[field]) {
      setErrors(prev => ({ ...prev, [field]: null }));
    }
  };

  const validateStep = (validators) => {
    const newErrors = {};
    Object.keys(validators).forEach(field => {
      const error = validators[field](formData[field]);
      if (error) newErrors[field] = error;
    });
    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  return {
    currentStep,
    formData,
    errors,
    nextStep,
    prevStep,
    updateField,
    validateStep,
    isLastStep: currentStep === totalSteps - 1,
    isFirstStep: currentStep === 0
  };
}
```

⚠️ Common Pitfalls

```
// ❌ Direct mutation (doesn't trigger re-render)
const [user, setUser] = useState({ name: 'Kashif' });
user.name = 'New Name'; // BAD!
```

```
// ✅ Immutable update
setUser(prev => ({ ...prev, name: 'New Name' }));

// ❌ Stale state in async callbacks
function handleClick() {
  setTimeout(() => {
    setCount(count + 1); // Uses count from render time, not current
  }, 1000);
}

// ✅ Functional update for async safety
function handleClick() {
  setTimeout(() => {
    setCount(prev => prev + 1); // Always uses latest state
  }, 1000);
}
```

`useReducer`: Complex State Logic

```
const [state, dispatch] = useReducer(reducer, initialArg, init?);
```

✅ When to Use

- State with multiple sub-values or complex transitions
- When next state depends on previous state in non-trivial ways
- When you want to separate state logic from component (testability)
- Forms with many fields, wizards, finite state machines

🔁 Reducer Pattern

```
// Reducer function: (currentState, action) => newState
function taskReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return [...state, { id: Date.now(), text: action.payload, completed: false }];
    case 'TOGGLE':
      return state.map(task =>
        task.id === action.payload
          ? { ...task, completed: !task.completed }
          : task
      );
    case 'DELETE':
      return state.filter(task => task.id !== action.payload);
    case 'SET_FILTER':
      return { ...state, filter: action.payload };
    case 'RESET':
      return action.payload; // initial state
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}
```

✅ Real-World Example: Shopping Cart with Reducer

```jsx
// contexts/CartContext.jsx
import { createContext, useContext, useReducer, useEffect } from 'react';

const CartContext = createContext();

const initialState = {
  items: [],
  discountCode: null,
  isLoading: false
};

function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existing = state.items.find(i => i.id === action.payload.id);
      if (existing) {
        return {
          ...state,
          items: state.items.map(i =>
            i.id === action.payload.id
              ? { ...i, quantity: i.quantity + 1 }
              : i
          )
        };
      }
      return {
        ...state,
        items: [...state.items, { ...action.payload, quantity: 1 }]
      };
    }
    case 'REMOVE_ITEM':
      return {
        ...state,
        items: state.items.filter(i => i.id !== action.payload)
      };
    case 'UPDATE_QUANTITY':
      return {
        ...state,
        items: state.items.map(i =>
          i.id === action.payload.id
            ? { ...i, quantity: action.payload.quantity }
            : i
        )
      };
    case 'APPLY_DISCOUNT':
      return { ...state, discountCode: action.payload };
    case 'CLEAR_CART':
      return initialState;
    case 'SET_LOADING':
      return { ...state, isLoading: action.payload };
    default:
      return state;
  }
}

export function CartProvider({ children }) {
  const [state, dispatch] = useReducer(cartReducer, initialState);

  // Persist cart to localStorage
  useEffect(() => {
    const saved = localStorage.getItem('cart');
```

```
    if (saved) {
      dispatch({ type: 'SET_ITEMS', payload: JSON.parse(saved) });
    }
  }, []);

  useEffect(() => {
    localStorage.setItem('cart', JSON.stringify(state.items));
  }, [state.items]);

  // Action creators for cleaner usage
  const actions = {
    addItem: (product) => dispatch({ type: 'ADD_ITEM', payload: product }),
    removeItem: (id) => dispatch({ type: 'REMOVE_ITEM', payload: id }),
    updateQuantity: (id, quantity) =>
      dispatch({ type: 'UPDATE_QUANTITY', payload: { id, quantity } }),
    applyDiscount: (code) => dispatch({ type: 'APPLY_DISCOUNT', payload: code }),
    clearCart: () => dispatch({ type: 'CLEAR_CART' })
  };

  return (
    <CartContext.Provider value={{ state, ...actions }}>
      {children}
    </CartContext.Provider>
  );
}

export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) throw new Error('useCart must be used within CartProvider');
  return context;
};
```

💡 Pro Tip: When to Choose `useReducer` Over `useState`

Use `useState`	Use `useReducer`
Simple boolean/number/string state	State with multiple related values
Independent state variables	State transitions depend on each other
Updates are straightforward	Complex update logic or multiple actions
No need to test state logic separately	Want to test state logic in isolation

---

2. Side Effect Hooks

`useEffect`: Declarative Side Effects

```
useEffect(() => {
  // Side effect code
  return () => {
    // Cleanup (optional)
  };
}, [dependencies]);
```

🔄 Dependency Array Patterns

```
// ✅ Run once on mount (like componentDidMount)
useEffect(() => {
```

```
  initializeAnalytics();
}, []);

// ✅ Run when specific values change (like componentDidUpdate)
useEffect(() => {
  fetchUser(userId);
}, [userId]);

// ✅ Run after every render (use sparingly!)
useEffect(() => {
  logRender();
});

// ✅ Cleanup on unmount or dependency change
useEffect(() => {
  const subscription = eventBus.subscribe('update', handleUpdate);
  return () => subscription.unsubscribe(); // Cleanup
}, [handleUpdate]);
```

✅ Real-World Example: API Sync with AbortController

```
// hooks/useApiSync.js
import { useState, useEffect } from 'react';

export function useApiSync(endpoint, options = {}) {
  const [data, setData] = useState(null);
  const [status, setStatus] = useState('idle'); // 'idle' | 'loading' | 'success' | 'error'
  const [error, setError] = useState(null);

  useEffect(() => {
    // Skip if no endpoint
    if (!endpoint) return;

    const controller = new AbortController();
    let isMounted = true;

    const fetchData = async () => {
      try {
        setStatus('loading');
        const response = await fetch(endpoint, {
          ...options,
          signal: controller.signal // Enable abort
        });

        if (!response.ok) {
          throw new Error(`HTTP ${response.status}: ${response.statusText}`);
        }

        const result = await response.json();

        if (isMounted) {
          setData(result);
          setStatus('success');
        }
      } catch (err) {
        if (isMounted && err.name !== 'AbortError') {
          setError(err.message);
          setStatus('error');
        }
      }
    };
```

```
    fetchData();

    // Cleanup: abort fetch and prevent state updates
    return () => {
      isMounted = false;
      controller.abort();
    };
  }, [endpoint, JSON.stringify(options)]); // Re-run when these change

  return { data, status, error, refetch: () => setStatus('idle') };
}
```

⚠️ Common Mistakes with `useEffect`

```
// ❌ Missing dependencies (ESLint will warn)
function UserProfile({ userId }) {
  useEffect(() => {
    fetch(`/api/users/${userId}`).then(setUser);
  }, []); // ❌ userId missing! Won't re-fetch when userId changes

  // ✅ Fix: Add to dependencies
  useEffect(() => {
    fetch(`/api/users/${userId}`).then(setUser);
  }, [userId]);
}

// ❌ Infinite loop: effect depends on object created every render
function Search({ filters }) {
  useEffect(() => {
    api.search(filters);
  }, [filters]); // ❌ filters is new object every render → infinite loop

  // ✅ Fix 1: Memoize the object
  const memoizedFilters = useMemo(() => filters, [filters.category, filters.price]);
  useEffect(() => {
    api.search(memoizedFilters);
  }, [memoizedFilters]);

  // ✅ Fix 2: Destructure primitives
  useEffect(() => {
    api.search({ category: filters.category, price: filters.price });
  }, [filters.category, filters.price]);
}
```

`useLayoutEffect`: Synchronous DOM Mutations

```
useLayoutEffect(() => {
  // Runs synchronously after DOM mutations, before paint
  return () => { /* cleanup */ };
}, [dependencies]);
```

✅ When to Use `useLayoutEffect` vs `useEffect`

Use `useEffect`	Use `useLayoutEffect`
Data fetching, subscriptions, logging	Measuring DOM nodes, synchronous re-renders

Use `useEffect`	Use `useLayoutEffect`
Most side effects (95% of cases)	When you need to mutate DOM before user sees paint
Async operations	Preventing visual flicker

✅ Real-World Example: Tooltip Positioning

```
// components/Tooltip.jsx
import { useRef, useLayoutEffect, useState } from 'react';

export default function Tooltip({ targetRef, content }) {
  const tooltipRef = useRef(null);
  const [position, setPosition] = useState({ top: 0, left: 0 });

  useLayoutEffect(() => {
    if (!targetRef.current || !tooltipRef.current) return;

    const targetRect = targetRef.current.getBoundingClientRect();
    const tooltipRect = tooltipRef.current.getBoundingClientRect();

    // Calculate position to appear above target, centered
    const top = targetRect.top - tooltipRect.height - 8; // 8px gap
    const left = targetRect.left + (targetRect.width / 2) - (tooltipRect.width / 2);

    // Keep tooltip within viewport
    const viewportWidth = window.innerWidth;
    const adjustedLeft = Math.max(8, Math.min(left, viewportWidth - tooltipRect.width - 8));

    setPosition({ top, left: adjustedLeft });
  }, [targetRef, content]); // Re-calculate when content changes (affects size)

  return (
    <div
      ref={tooltipRef}
      className="tooltip"
      style={{
        position: 'fixed',
        top: `${position.top}px`,
        left: `${position.left}px`,
        // Prevent flash: start hidden, fade in after position set
        opacity: 0,
        animation: 'fadeIn 0.2s forwards'
      }}
    >
      {content}
    </div>
  );
}
```

> ⚠️ **Warning:** Prefer `useEffect` unless you see visual flicker. `useLayoutEffect` blocks painting and can hurt performance if overused.

---

3. Performance Optimization Hooks

`useCallback`: Memoize Functions

```
const memoizedCallback = useCallback(() => {
  // function body
}, [dependencies]);
```

✅ When to Use

- Passing callbacks to optimized child components that rely on reference equality (`React.memo`)
- Functions used as dependencies in other hooks (`useEffect` , `useMemo`)
- Expensive function creation (rare—measure first!)

✅ Real-World Example: Optimized List with Memoized Handlers

```
// components/ProductList.jsx
import { memo, useCallback, useState } from 'react';
import ProductItem from './ProductItem';

// Child component - won't re-render if onAddToCart reference doesn't change
const ProductItemMemo = memo(ProductItem);

export default function ProductList({ products, onAddToCart }) {
  const [selectedCategory, setSelectedCategory] = useState('all');

  // ❌ BAD: New function created every render → ProductItem re-renders unnecessarily
  // const handleAdd = (product) => onAddToCart(product);

  // ✅ GOOD: Memoized - same reference if dependencies unchanged
  const handleAddToCart = useCallback((product) => {
    // Add analytics, validation, etc.
    console.log(`Adding ${product.name} to cart`);
    onAddToCart(product);
  }, [onAddToCart]); // Re-create only if onAddToCart changes

  // Filter products (could also use useMemo if expensive)
  const filteredProducts = selectedCategory === 'all'
    ? products
    : products.filter(p => p.category === selectedCategory);

  return (
    <div>
      <CategoryFilter
        selected={selectedCategory}
        onChange={setSelectedCategory}
      />
      <div className="product-grid">
        {filteredProducts.map(product => (
          <ProductItemMemo
            key={product.id}
            product={product}
            onAddToCart={handleAddToCart} // Stable reference!
          />
        ))}
      </div>
    </div>
  );
}
```

⚠️ Don't Overuse `useCallback`

```
// ❌ Unnecessary: function isn't passed to memoized child or used as dependency
function MyComponent() {
  const [count, setCount] = useState(0);

  // This creates overhead with no benefit
  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []);

  return <button onClick={handleClick}>Click</button>;
}

// ✅ Better: Simple inline function is fine here
function MyComponent() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>Click</button>;
}
```

> 🎯 **Rule of Thumb:** Only use `useCallback` when:
>
> 1. Passing to a `React.memo`-wrapped child, OR
> 2. Used in a `useEffect` / `useMemo` dependency array AND the function is expensive to create

---

`useMemo`: Memoize Expensive Calculations

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

✅ When to Use

- Expensive computations (filtering/sorting large lists, data transformation)
- Creating objects/arrays that are dependencies for other hooks
- Preventing re-creation of heavy objects on every render

✅ Real-World Example: Filtered & Sorted Dashboard Data

```
// components/AnalyticsDashboard.jsx
import { useMemo, useState } from 'react';

export default function AnalyticsDashboard({ rawEvents, dateRange }) {
  const [sortBy, setSortBy] = useState('date');
  const [filterType, setFilterType] = useState('all');

  // Expensive computation: filter + sort + aggregate
  const processedData = useMemo(() => {
    console.log('🔄 Recomputing analytics...'); // Log to see when this runs

    // 1. Filter by date range
    const filtered = rawEvents.filter(event => {
      const eventDate = new Date(event.timestamp);
      return eventDate >= dateRange.start && eventDate <= dateRange.end;
    });

    // 2. Filter by type
    const typeFiltered = filterType === 'all'
      ? filtered
      : filtered.filter(e => e.type === filterType);
```

```
    // 3. Sort (expensive on large datasets)
    const sorted = [...typeFiltered].sort((a, b) => {
      if (sortBy === 'date') return new Date(b.timestamp) - new Date(a.timestamp);
      if (sortBy === 'value') return b.value - a.value;
      return 0;
    });

    // 4. Aggregate metrics
    const metrics = {
      total: sorted.length,
      sum: sorted.reduce((acc, e) => acc + e.value, 0),
      avg: sorted.length > 0
        ? sorted.reduce((acc, e) => acc + e.value, 0) / sorted.length
        : 0,
      byType: sorted.reduce((acc, e) => {
        acc[e.type] = (acc[e.type] || 0) + 1;
        return acc;
      }, {})
    };

    return { events: sorted, metrics };
  }, [rawEvents, dateRange, sortBy, filterType]); // Re-compute only when these change

  // This re-renders often, but processedData won't recompute unnecessarily
  return (
    <div>
      <Controls
        sortBy={sortBy}
        onSortChange={setSortBy}
        filterType={filterType}
        onFilterChange={setFilterType}
      />

      <MetricsSummary metrics={processedData.metrics} />

      <EventTable events={processedData.events} />
    </div>
  );
}
```

⚠️ Common Pitfalls

```
// ❌ Memoizing primitive values (no benefit)
const count = useMemo(() => 42, []); // Just use: const count = 42;

// ❌ Memoizing without dependencies (runs once, may be stale)
const data = useMemo(() => compute(apiData), []); // Won't update when apiData changes!

// ✅ Correct: Include all dependencies
const data = useMemo(() => compute(apiData), [apiData]);

// ❌ Over-memoizing simple operations
const fullName = useMemo(() => `${firstName} ${lastName}`, [firstName, lastName]);
// String concatenation is cheap—just compute directly:
const fullName = `${firstName} ${lastName}`;
```

> 🎯 **Rule of Thumb:** Use `useMemo` when:
>
> 1. The computation is **expensive** (benchmark if unsure), OR
> 2. You need **reference equality** for an object/array used in dependencies or `React.memo`

4. Reference & Context Hooks

`useRef` : Mutable References Without Re-renders

```
const refContainer = useRef(initialValue);
// Access via: refContainer.current
```

✅ When to Use

- Accessing DOM elements directly (focus, measurements, animations)
- Storing mutable values that shouldn't trigger re-renders
- Keeping previous values for comparison (`usePrevious` pattern)
- Storing IDs for intervals/timeouts (for cleanup)

✅ Real-World Examples

Example 1: Focus Management

```
// components/LoginForm.jsx
import { useRef, useEffect } from 'react';

export default function LoginForm({ showError }) {
  const emailRef = useRef(null);
  const passwordRef = useRef(null);

  // Auto-focus email field on mount
  useEffect(() => {
    emailRef.current?.focus();
  }, []);

  // Focus password if email has error
  useEffect(() => {
    if (showError?.field === 'email') {
      emailRef.current?.focus();
    } else if (showError?.field === 'password') {
      passwordRef.current?.focus();
    }
  }, [showError]);

  return (
    <form>
      <input ref={emailRef} name="email" type="email" />
      <input ref={passwordRef} name="password" type="password" />
    </form>
  );
}
```

Example 2: Previous Value Hook (Custom Hook)

```
// hooks/usePrevious.js
import { useRef, useEffect } from 'react';

export function usePrevious(value) {
  const ref = useRef();

  useEffect(() => {
    ref.current = value; // Update after render
```

```
  }, [value]);

  return ref.current; // Returns value from PREVIOUS render
}

// Usage:
function Counter() {
  const [count, setCount] = useState(0);
  const prevCount = usePrevious(count);

  return (
    <div>
      <p>Now: {count}, Before: {prevCount}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
  );
}
```

Example 3: Interval ID for Cleanup

```
function AutoSave({ content, onSave }) {
  const intervalRef = useRef(null);

  useEffect(() => {
    // Start auto-save interval
    intervalRef.current = setInterval(() => {
      onSave(content);
    }, 30000); // Every 30 seconds

    // Cleanup on unmount or content change
    return () => {
      if (intervalRef.current) {
        clearInterval(intervalRef.current);
      }
    };
  }, [content, onSave]);

  return <Editor value={content} />;
}
```

> ⚠️ **Important:** Changing `ref.current` does **not** trigger a re-render. Use `useState` when you need UI updates.

`useContext` : Consume Context Values

```
const value = useContext(MyContext);
```

✅ When to Use

- Avoiding prop drilling for globally needed data (theme, auth, locale)
- Sharing state across deeply nested components
- When combined with `useReducer` for global state management

✅ Real-World Pattern: Theme + Auth Context Composition

```
// contexts/AppContext.jsx
import { createContext, useContext, useMemo } from 'react';
import { useTheme } from '../hooks/useTheme';
```

```
import { useAuth } from '../hooks/useAuth';

// Create a combined context for easy consumption
const AppContext = createContext(null);

export function AppProvider({ children }) {
  const theme = useTheme(); // Custom hook using useState/useEffect
  const auth = useAuth();   // Custom hook with useReducer/useEffect

  // Memoize the context value to prevent unnecessary re-renders
  const value = useMemo(() => ({
    theme: theme.theme,
    toggleTheme: theme.toggleTheme,
    user: auth.user,
    login: auth.login,
    logout: auth.logout,
    isLoading: auth.loading
  }), [theme, auth]);

  return (
    <AppContext.Provider value={value}>
      {children}
    </AppContext.Provider>
  );
}

// Single hook to access all app-level state
export const useApp = () => {
  const context = useContext(AppContext);
  if (!context) {
    throw new Error('useApp must be used within AppProvider');
  }
  return context;
};
```

```
// components/Header.jsx - Clean consumption
import { useApp } from '../contexts/AppContext';

export default function Header() {
  const { theme, toggleTheme, user, logout } = useApp();

  return (
    <header className={`header header-${theme}`}>
      <Logo />

      {user ? (
        <div className="user-menu">
          <span>Welcome, {user.name}</span>
          <button onClick={logout}>Sign Out</button>
        </div>
      ) : (
        <AuthButtons />
      )}

      <ThemeToggle
        currentTheme={theme}
        onToggle={toggleTheme}
      />
    </header>
  );
}
```

⚠️ Performance Tip: Context Re-renders

```
// ❌ BAD: Context value changes every render → all consumers re-render
function App() {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState('light');

  // New object every time → triggers re-renders even if values unchanged
  const value = { user, setUser, theme, setTheme };

  return <AppContext.Provider value={value}><Main /></AppContext.Provider>;
}

// ✅ GOOD: Memoize the context value
function App() {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState('light');

  const value = useMemo(() => ({
    user, setUser, theme, setTheme
  }), [user, theme]); // Only re-create when dependencies change

  return <AppContext.Provider value={value}><Main /></AppContext.Provider>;
}

// ✅ BETTER: Split contexts by update frequency
const UserContext = createContext();
const ThemeContext = createContext();

function App() {
  const userState = useState(null);
  const themeState = useState('light');

  return (
    <UserContext.Provider value={userState}>
      <ThemeContext.Provider value={themeState}>
        <Main />
      </ThemeContext.Provider>
    </UserContext.Provider>
  );
}
// Now ThemeToggle only re-renders when theme changes, not when user changes
```

---

5. Advanced & Specialized Hooks

`useImperativeHandle` : Customize Ref Exposure

```
useImperativeHandle(ref, () => ({
  // Methods to expose to parent
  focus: () => { /* ... */ },
  validate: () => { /* ... */ }
}), [dependencies]);
```

✅ When to Use

- Building reusable component libraries
- When a parent needs to call child methods (rare—prefer state/props first)

- Integrating with non-React code

✅ Example: Custom Input with Exposed Methods

```
// components/ValidatedInput.jsx
import { useRef, useImperativeHandle, forwardRef } from 'react';

const ValidatedInput = forwardRef(function ValidatedInput(
  { label, validator, ...props },
  ref
) {
  const inputRef = useRef(null);

  // Expose custom methods to parent via ref
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current?.focus(),
    validate: () => {
      const value = inputRef.current?.value || '';
      const error = validator?.(value);
      return { isValid: !error, error };
    },
    getValue: () => inputRef.current?.value || ''
  }), [validator]);

  return (
    <div className="form-field">
      <label>{label}</label>
      <input ref={inputRef} {...props} />
    </div>
  );
});

// Usage in parent
function SignupForm() {
  const emailRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();

    const validation = emailRef.current?.validate();
    if (!validation?.isValid) {
      alert(validation.error);
      emailRef.current?.focus();
      return;
    }

    // Proceed with submission...
  };

  return (
    <form onSubmit={handleSubmit}>
      <ValidatedInput
        ref={emailRef}
        label="Email"
        type="email"
        validator={(value) =>
          !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)
            ? 'Invalid email format'
            : null
        }
      />
```

```
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

> ⚠️ **Caution:** This breaks React's data flow. Use only when necessary (e.g., for library components).

`useDebugValue` : Label Custom Hooks in DevTools

```
useDebugValue(value, formatFn?);
```

✅ When to Use

- Building custom hooks you want to debug easily
- Library development for better developer experience

✅ Example: Enhancing a Custom Hook

```
// hooks/useAuth.js
import { useState, useEffect, useDebugValue } from 'react';

export function useAuth() {
  const [user, setUser] = useState(null);
  const [status, setStatus] = useState('checking'); // 'checking' | 'authenticated' |
'unauthenticated'

  // Add debug label visible in React DevTools
  useDebugValue(status, s => `Auth: ${s}`);

  useEffect(() => {
    // ... auth logic
    setStatus('authenticated'); // This label appears in DevTools
  }, []);

  return { user, status, login, logout };
}
```

In React DevTools, you'll see:
`useAuth → Auth: authenticated` instead of just raw state.

Custom Hooks: The Ultimate Abstraction

> "Custom hooks are how you extract component logic into reusable functions." — React Docs

✅ Pattern: Extracting Feature Logic

```
// hooks/useLocalStorage.js
import { useState, useEffect } from 'react';

export function useLocalStorage(key, initialValue) {
  // Get initial value from localStorage or fallback
  const [storedValue, setStoredValue] = useState(() => {
    if (typeof window === 'undefined') return initialValue;

    try {
```

```
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.warn(`Error reading localStorage key "${key}":`, error);
      return initialValue;
    }
  });

  // Update localStorage when value changes
  useEffect(() => {
    try {
      if (storedValue === undefined) {
        localStorage.removeItem(key);
      } else {
        localStorage.setItem(key, JSON.stringify(storedValue));
      }
    } catch (error) {
      console.warn(`Error setting localStorage key "${key}":`, error);
    }
  }, [key, storedValue]);

  // Return state + setter (with error handling)
  const setValue = (value) => {
    try {
      // Allow function updates like useState
      const valueToStore = value instanceof Function
        ? value(storedValue)
        : value;
      setStoredValue(valueToStore);
    } catch (error) {
      console.warn(`Error updating localStorage key "${key}":`, error);
    }
  };

  return [storedValue, setValue];
}
```

✅ Usage: Clean, Reusable, Testable

```
// components/UserPreferences.jsx
import { useLocalStorage } from '../hooks/useLocalStorage';

export default function UserPreferences() {
  const [theme, setTheme] = useLocalStorage('user-theme', 'light');
  const [fontSize, setFontSize] = useLocalStorage('user-font-size', 16);

  return (
    <div>
      <select value={theme} onChange={(e) => setTheme(e.target.value)}>
        <option value="light">Light</option>
        <option value="dark">Dark</option>
      </select>

      <input
        type="range"
        min="12"
        max="24"
        value={fontSize}
        onChange={(e) => setFontSize(Number(e.target.value))}
      />
    </div>
```

```
  );
}
```

> 🎯 **Custom Hook Checklist:**
>
> - ✅ Name starts with `use`
> - ✅ Only calls other hooks (no regular JS functions)
> - ✅ Document inputs/outputs with JSDoc
> - ✅ Handle edge cases (SSR, errors, undefined values)
> - ✅ Write unit tests for the hook logic

⚠️ Rules of Hooks (Recap)

1. **Only call hooks at the top level**
   → Never inside loops, conditions, or nested functions
2. **Only call hooks from React functions**
   → Components or custom hooks (not regular JS functions)
3. **Enable ESLint plugin**

```
npm install eslint-plugin-react-hooks --save-dev

// .eslintrc.js
rules: {
  'react-hooks/rules-of-hooks': 'error',
  'react-hooks/exhaustive-deps': 'warn'
}
```

4. **Respect dependency arrays**
   → If you use a variable inside `useEffect` / `useMemo` / `useCallback`, it belongs in the deps array (unless you intentionally want stale values—and document why)

🎯 Best Practices & Anti-Patterns

✅ Do This

Practice	Example
Extract logic early	Turn repeated `useEffect` patterns into `useFetch`, `useForm`, etc.
Memoize context values	`useMemo(() => ({ user, theme }), [user, theme])`
Use functional updates	`setCount(prev => prev + 1)` for async safety
Name custom hooks clearly	`useAuth`, `useCart`, `useMediaQuery`—self-documenting
Test custom hooks in isolation	They're just functions—use React Testing Library

❌ Avoid This

```
// ❌ Anti-pattern: Over-memoization
function OverOptimized({ items, filter }) {
  // Unnecessary: filter function is cheap
  const filtered = useMemo(() =>
    items.filter(i => i.active),
    [items]
  );
```

```
  // Unnecessary: handler not passed to memoized child
  const handleClick = useCallback(() => {
    console.log('clicked');
  }, []);

  return <div onClick={handleClick}>{filtered.length}</div>;
}

// ✅ Better: Optimize only when needed
function JustRight({ items, filter }) {
  const filtered = items.filter(i => i.active); // Cheap enough

  return <div>{filtered.length}</div>;
}
```

```
// ❌ Anti-pattern: Stale closures in effects
function Chat({ roomId }) {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    const connection = createConnection(roomId);
    connection.on('message', (msg) => {
      // ❌ Uses messages from render time, not current
      setMessages([...messages, msg]);
    });
    return () => connection.close();
  }, [roomId]); // messages missing → stale closure

  // ✅ Fix 1: Functional update
  useEffect(() => {
    const connection = createConnection(roomId);
    connection.on('message', (msg) => {
      setMessages(prev => [...prev, msg]); // Always latest
    });
    return () => connection.close();
  }, [roomId]);

  // ✅ Fix 2: Add to dependencies (may re-subscribe unnecessarily)
  useEffect(() => {
    const connection = createConnection(roomId);
    connection.on('message', (msg) => {
      setMessages([...messages, msg]);
    });
    return () => connection.close();
  }, [roomId, messages]); // Re-runs on every message → may not be desired
}
```

🧭 Decision Flowchart: Which Hook to Use?

```
Need to store data that affects UI?
├─ Simple value? → useState
└─ Complex transitions? → useReducer

Need to run code after render?
├─ Async/API/subscriptions? → useEffect
└─ Sync DOM measurements? → useLayoutEffect

Passing callback to memoized child?
└─ Yes → useCallback
```

```
Computing expensive value?
└─ Yes → useMemo

Need DOM access or mutable value without re-render?
└─ Yes → useRef

Sharing data across many components?
└─ Yes → useContext + Context.Provider

Building reusable logic?
└─ Yes → Custom Hook (combine above)
```

🧪 Lab Activity: Build a Real-Time Dashboard

🎯 Goal

Create a dashboard that fetches live data, allows filtering, persists preferences, and optimizes performance using multiple hooks.

📦 Starter Structure

```
src/
├── hooks/
│   ├── useApi.js          # Custom fetch hook with caching
│   ├── useLocalStorage.js # Persistent state
│   └── useDebounce.js     # Input optimization
├── contexts/
│   └── DashboardContext.jsx # Shared state with useReducer
├── components/
│   ├── DataChart.jsx      # Memoized visualization
│   ├── FilterPanel.jsx    # Controlled form with useCallback
│   └── StatusBar.jsx      # Context consumer with useRef
└── App.jsx
```

📋 Progressive Tasks

✅ Task 1: Data Fetching with `useApi` (20 mins)

```javascript
// hooks/useApi.js (STARTER)
import { useState, useEffect, useCallback } from 'react';

export function useApi(endpoint, options = {}) {
  const [data, setData] = useState(null);
  const [status, setStatus] = useState('idle');
  const [error, setError] = useState(null);

  const fetch = useCallback(async () => {
    // TODO: Implement fetch with AbortController
    // TODO: Handle loading/error/success states
    // TODO: Return cleanup function
  }, [endpoint, JSON.stringify(options)]);

  useEffect(() => {
    fetch();
  }, [fetch]);
```

```
  return { data, status, error, refetch: fetch };
}
```

Deliverable: A `useApi` hook that safely fetches JSON data with loading states and cleanup.

✅ Task 2: Persistent Filters with `useLocalStorage` (15 mins)

```
// components/FilterPanel.jsx
import { useLocalStorage } from '../hooks/useLocalStorage';

export default function FilterPanel({ onFilterChange }) {
  // TODO: Persist category, dateRange, and sortBy to localStorage
  // TODO: Sync local state with localStorage using custom hook
  // TODO: Notify parent via onFilterChange when filters update

  return (
    <div className="filters">
      {/* Filter controls */}
    </div>
  );
}
```

Deliverable: Filters that survive page refresh and sync with parent component.

✅ Task 3: Optimized Chart Rendering (25 mins)

```
// components/DataChart.jsx
import { useMemo, memo } from 'react';

const DataChart = memo(function DataChart({ rawData, config }) {
  // TODO: Use useMemo to process rawData → chartData only when needed
  // TODO: Memoize chart configuration object
  // TODO: Prevent re-renders when unrelated props change

  return <canvas>{/* Render chart */}</canvas>;
});

export default DataChart;
```

Deliverable: A chart component that only re-renders when its specific data/config changes.

🚀 Challenge Task: Real-Time Updates with `useRef` (Bonus)

- Add WebSocket support to `useApi`
- Use `useRef` to store the WebSocket instance (no re-renders on connect)
- Use `useEffect` to manage connection lifecycle
- Use functional updates to append new messages without stale closures

🎯 Success Criteria

- ✅ All hooks follow the Rules of Hooks (ESLint clean)
- ✅ No unnecessary re-renders (verify with React DevTools Profiler)
- ✅ Custom hooks are tested in isolation
- ✅ Preferences persist across sessions
- ✅ Code is documented with JSDoc for custom hooks

💡 Starter Tips

1. Use `JSON.stringify` carefully in dependency arrays—memoize objects first
2. Wrap context values in `useMemo` to prevent consumer re-renders
3. Test custom hooks with a simple test component before integrating
4. Use React DevTools → Components → Highlight updates to spot performance issues

---

📚 Reference Cheat Sheet

Hook Quick Lookup

Hook	Syntax	Key Dependencies	Common Use Case
`useState`	`[val, setVal] = useState(init)`	None	Local component state
`useReducer`	`[state, dispatch] = useReducer(fn, init)`	Reducer function	Complex state logic
`useEffect`	`useEffect(() => { ... }, [deps])`	Dependencies array	API calls, subscriptions
`useLayoutEffect`	Same as useEffect	Same	DOM measurements
`useCallback`	`fn = useCallback(() => {}, [deps])`	Dependencies	Memoized event handlers
`useMemo`	`val = useMemo(() => compute(), [deps])`	Dependencies	Expensive calculations
`useRef`	`ref = useRef(init)`	None	DOM access, mutable values
`useContext`	`val = useContext(Context)`	Context object	Global state consumption
`useImperativeHandle`	`useImperativeHandle(ref, fn, [deps])`	Ref + dependencies	Library component APIs
`useDebugValue`	`useDebugValue(val, format)`	None	Custom hook debugging

Dependency Array Guidelines

```
// ✅ Include:
- Variables used inside the effect/callback
- Props/state that affect the logic
- Functions that aren't memoized (or memoize them first!)

// ❌ Exclude:
- Functions already wrapped in useCallback with correct deps
- Values that never change (constants, module imports)
- Values you intentionally want to "ignore" (document why!)

// 🔍 When in doubt:
1. Let ESLint `exhaustive-deps` warn you
2. Ask: "If this value changed, should the effect re-run?"
3. Test with React.StrictMode (double-invokes effects in dev)
```

🚀 Next Steps & Resources

🔗 Essential Links

- React Hooks API Reference
- Rules of Hooks
- React DevTools Profiler Guide
- useHooks.com - Production-ready custom hooks

📖 Recommended Reading

- *"Learning React"* by Alex Banks & Eve Porcello (Hooks chapters)
- Kent C. Dodds: Fix the slow render before you fix the re-render
- Dan Abramov: A Complete Guide to useEffect

🛠️ Practice Projects

1. Refactor a class component portfolio piece to use hooks
2. Build a custom `useForm` hook with validation
3. Create a `useWebSocket` hook with auto-reconnect
4. Optimize a slow list component with `React.memo` + `useCallback`

> 💬 **Instructor Closing Note:**
> Hooks are a mindset shift—not just new APIs. You're no longer thinking in lifecycle methods, but in **synchronizing state with effects**. The patterns we covered today (fetching, forms, context, optimization) are the foundation of professional React development.
>
> **Your challenge this week**: Audit one of your projects. Find one component that:
>
> - Uses a class → refactor to hooks
> - Has prop drilling → extract a custom hook or context
> - Re-renders unnecessarily → apply `useMemo` / `useCallback` *with measurement*
>
> Share your before/after on LinkedIn with `#MERNBootcamp` —I'll feature the best improvements!
>
> Remember: Optimize for readability first, performance second. Hooks make both possible.
>
> — Kashif Raza | Writing clean hooks, building robust systems 🪝

© 2025 MERN Stack Bootcamp. For educational use only. Code examples are MIT licensed.